SOM + RBF

Hybrid Neural Network Project

Customer Segmentation + Spending Score Prediction System

Submitted by	Debasmita Banerjee
University	Bhawanipur Global Campus
Department	Computer Science / IT
Course	Bsc. DataScience
Year	2nd Year
Date	10 May 2026

Abstract

This project presents a Hybrid Neural Network system that combines a Self-Organizing Map (SOM) with a Radial Basis Function (RBF) Network to perform customer segmentation and spending score prediction on the Mall Customers dataset. The SOM operates as an unsupervised learning stage to discover the natural cluster structure within customer data, mapping 200 customers across a 5x5 neuron grid. The trained neuron weight vectors are then extracted and used directly as the radial basis centers for the RBF network, replacing the conventional random center selection method. A Linear Regression model is placed on top of the RBF feature matrix to predict continuous Spending Scores. The model achieves an R^2 score of 0.571 and RMSE of 14.54 on the test set, demonstrating meaningful predictive capability from only two customer attributes. This report documents the complete methodology, implementation, results, and key insights gained through the project.

1. Problem Statement

1.1 Business Context

Shopping malls generate large volumes of customer data from loyalty programs and purchase histories. A business challenge commonly faced by mall management and retail marketing teams is: given basic customer information such as age and annual income, can we automatically group customers into meaningful segments and then predict their spending behaviors?

This has two distinct sub-problems that require two different types of learning:

- Segmentation (unsupervised): Discover natural customer groups without pre-defined labels.
- Prediction (supervised): Use those discovered groups to predict a continuous target — the Spending Score (1–100).

1.2 Formal Problem Definition

Element	Description
Dataset	Mall Customers (200 rows, 5 columns)
Input Features	Age, Annual Income (k$), SOM Cluster ID
Target Variable	Spending Score (1–100) — a continuous value
Constraint	Spending Score must NOT be used as an input (data leakage)
Goal	Segment customers via SOM, then predict Spending Score via RBF

1.3 Why This Problem Matters

- Marketing teams can personalize promotions based on predicted spending behavior.
- High-income but low-spending customers can be targeted with specific incentive campaigns.
- Cluster-based models are more interpretable than black-box approaches like deep learning.

2. Background and Theory

2.1 Self-Organizing Map (SOM)

A Self-Organizing Map, introduced by Teuvo Kohonen (1982), is an unsupervised artificial neural network that learns a low-dimensional, topology-preserving representation of the input space. Unlike standard neural networks that minimise a loss function via backpropagation, the SOM uses competitive learning.

2.1.1 Architecture

A SOM consists of a 2D grid of neurons. Each neuron i holds a weight vector w_i of the same dimension as the input data. During training, input vectors compete to be the "best match" and the winner updates its neighbors.

2.1.2 Training Algorithm

1. Present an input sample x from the dataset.
2. Find the Best Matching Unit (BMU): the neuron whose weight vector is closest to x in Euclidean distance.
3. Update the BMU and all neighboring neurons using the rule:

```
w_i(t+1) = w_i(t) + lr(t) x h(d_grid, t) x (x - w_i(t))

where:
  lr(t)        = learning rate at epoch t (decays over time)
  h(d_grid, t) = Gaussian neighborhood function
  d_grid       = grid distance between neuron i and the BMU
```

4. Reduce learning rate and neighborhood radius. Repeat.

2.1.3 Key Parameters Used

Parameter	Value	Meaning
Grid Size	5 x 5	25 neurons arranged in a grid
input_len	3	Age, Income, Spending Score (for SOM phase)
sigma	1.0	Initial neighborhood radius

learning_rate	0.5	Initial step size for weight updates
Iterations	100	Number of random samples shown during training

2.2 Radial Basis Function (RBF) Network

An RBF Network is a three-layer feedforward network with a special hidden layer that uses radial (distance-based) activation functions instead of sigmoid or ReLU. It was popularized by Broomhead & Lowe (1988) and has strong universal approximation properties.

2.2.1 Architecture

The three layers are:

- Input layer: Passes raw features through unchanged.
- Hidden layer (RBF layer): Each neuron computes a Gaussian kernel activation based on distance to its assigned center.
- Output layer: A linear weighted sum of the hidden activations. Weights here are learned analytically.

2.2.2 Gaussian Kernel Formula

```
φ_k(x) = exp( -γ × ||x - c_k||^2 )

where:
  x    = input data point
  c_k  = center of the k-th RBF neuron
  γ    = gamma parameter (controls kernel width)
  ||.|| = Euclidean distance

When x is close to c_k  →  φ_k(x) ≈ 1  (high activation)
When x is far from c_k  →  φ_k(x) ≈ 0  (low activation)
```

In this project, gamma = 5.0 was used. Higher gamma values make the Gaussian bell narrower, meaning only very close points get strong activations from a center.

2.3 The Hybrid Approach: Why Combine SOM + RBF?

The critical design question in any RBF network is: where do the centers come from? Common choices are:

Method	Pros	Cons

Random selection	Fast	May miss important data regions
k-means clustering	Good coverage	Extra algorithm, hyperparameter k
SOM neurons (this project)	Topology-preserving, principled placement	Slightly more setup

Using SOM neurons as RBF centers is theoretically superior because the SOM has already learned which regions of input space are densely populated. Each neuron represents a prototype for a region of the data manifold, making it a principled and data-driven choice for RBF center placement.

3. Dataset Description

3.1 Overview

The Mall Customers dataset is a standard benchmark dataset widely used for clustering and segmentation demonstrations. It was made available on Kaggle and simulates customer data from a shopping mall.

Column	Type	Description
CustomerID	Integer	Unique identifier (dropped — not informative)
Gender	String	Male / Female (not used in this model)
Age	Integer	Age of customer (18–70)
Annual Income (k$)	Integer	Annual income in thousands of dollars (15–137)
Spending Score (1-100)	Integer	Mall-assigned score; TARGET variable

3.2 Preprocessing

3.2.1 Feature Selection

Three numeric columns were selected for SOM training: Age, Annual Income, and Spending Score. CustomerID carries no pattern. Gender was excluded to keep the model simple; it could be one-hot encoded in a future version.

3.2.2 MinMax Scaling

All three features were scaled to [0, 1] using MinMaxScaler before SOM training. This is critical because SOM uses Euclidean distance to find the BMU. Without scaling, Annual Income (range: 15–137) would completely dominate Age (range: 18–70) and Spending Score (range: 1–100), making the distance calculation meaningless.

```
scaler = MinMaxScaler()
x_scaled = scaler.fit_transform(x)

# Example: Annual Income 137 -> 1.0
#          Annual Income  15 -> 0.0
#          Age            19 -> 0.018
#          Age            70 -> 1.0
```

3.2.3 Data Leakage Prevention

When preparing features for the RBF prediction model, Spending Score was excluded from the input features (X_rbf). Including it would create data leakage: the model would achieve artificially perfect accuracy during training but would completely fail on real-world data where the spending score is not known in advance. This is a common mistake in student ML projects; avoiding it here demonstrates proper ML methodology.

4. Methodology

4.1 Pipeline Overview

The project follows a clear 6-stage pipeline where the output of Stage 2 directly feeds into Stage 3, creating the hybrid connection:

Stage	Component	Description
1	Data Loading	Read Mall_Customers.csv, explore shape and types
2	Preprocessing	Select features, scale to [0,1] with MinMaxScaler
3	SOM Training	Train 5×5 SOM on scaled data; visualize U-Matrix
4	Center Extraction	Extract 25 neuron weight vectors as RBF centers (KEY STEP)
5	RBF Transformation	Apply Gaussian kernel to create 200×25 feature matrix
6	Model Training	Train LinearRegression on RBF features; evaluate on test set

4.2 Stage 3: SOM Training

The SOM is created with MiniSom, a lightweight Python library. The 5x5 grid was chosen to provide 25 prototype neurons, which is sufficient to represent the 5 known customer clusters in this dataset while avoiding over-segmentation.

```
som = MiniSom(x=5, y=5, input_len=3, sigma=1.0, learning_rate=0.5)
som.random_weights_init(x_scaled)   # init weights from data range
som.train_random(x_scaled, 100)     # 100 iterations
```

After training, each customer is mapped to their winning neuron and assigned a cluster ID by converting the 2D grid position (i, j) to a flat integer: cluster_id = i x 5 + j. This produces cluster IDs from 0 to 24.

4.3 Stage 4: Center Extraction (The Hybrid Connection)

This is the most important step of the entire project. Instead of selecting random data points as RBF centers, the trained SOM neuron weight vectors are extracted directly:

```
# som.get_weights() returns shape (5, 5, 3)
# reshape(-1, 3) flattens to (25, 3)
centers = som.get_weights().reshape(-1, 3)

# Result: 25 center vectors, each representing
# a region of the (Age, Income, SpendingScore) space
# that the SOM learned during unsupervised training.
```

These 25 centers have been optimally placed by the SOM to represent the structure of the data. This gives the RBF network a much stronger foundation than randomly chosen centers.

4.4 Stage 5: RBF Feature Transformation

The RBF transformation converts each data point into a 25-dimensional vector where each element represents how close that customer is to one of the 25 SOM-derived centers:

```
from scipy.spatial.distance import cdist

def rbf(X, centers, gamma=5.0):
    # cdist computes ||x_i - c_k|| for all i, k
    # Result shape: (200, 25)
    return np.exp(-gamma * cdist(X, centers)**2)

X_features = rbf(X_rbf_scaled, centers)
# X_features shape: (200, 25)
# Each row = one customer's RBF activation profile
```

4.5 Stage 6: Prediction Model

A Linear Regression model is trained on the RBF feature matrix to predict Spending Scores. This completes the classic RBF Network architecture: a non-linear hidden transformation followed by a linear output layer.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    X_features, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(x_train, y_train)
predictions = model.predict(x_test)
```

5. Results and Evaluation

5.1 SOM Results

After 100 training iterations, the SOM produced a well-organized distance map (U-Matrix). The U-Matrix shows regions of low inter-neuron distance (dense clusters in blue/purple) surrounded by boundary regions of high distance (in red). This confirms that the SOM successfully identified the natural groupings in customer data without any labels.

The cluster assignment step assigned each of the 200 customers to one of the 25 neurons, producing meaningful customer segments. Neurons with higher concentrations of data points in specific regions of the U-Matrix correspond to distinct customer profiles.

5.2 RBF Prediction Results

Metric	Value	Interpretation
MSE	211.53	Average squared prediction error
RMSE	14.54	Predictions off by ~15 score points on average
R^2 Score	0.571	Model explains 57.1% of spending variance

Test Size	40 samples	20% of 200-row dataset

5.3 Interpretation of Results

An R^2 of 0.571 means the model explains approximately 57% of the variance in spending scores. While not near-perfect, this is a meaningful result given that we are predicting from only two raw customer attributes (Age and Annual Income). Spending behavior in real life is influenced by many factors not captured in this dataset, such as personality, lifestyle, and marketing exposure.

The Actual vs Predicted scatter plot shows a clear positive trend along the ideal diagonal line, confirming that the model has learned a genuine relationship. The predictions are most accurate for customers with mid-range spending scores (40–60); high and low extremes are harder to predict, which is expected.

The RMSE of 14.54 means that, on average, the model's prediction is about 14-15 points away from the true Spending Score on the 1-100 scale. For marketing applications, this level of accuracy is often sufficient to categorize customers into broad spending tiers (low, medium, high).

5.4 Key Observations

- The SOM successfully organized customers into topologically meaningful groups.
- Using SOM neuron weights as RBF centers created a principled, data-driven basis for the RBF transformation.
- Excluding Spending Score from input features was essential to prevent data leakage and ensure genuine generalization.
- The hybrid approach is more interpretable than a black-box model: each SOM neuron represents a customer archetype, and the RBF activation shows how strongly each archetype contributes to the final prediction.

6. Conclusion

6.1 Summary

This project successfully demonstrated a Hybrid Neural Network pipeline combining an unsupervised Self-Organizing Map with a supervised Radial Basis Function Network for the task of customer segmentation and spending score prediction. The key innovation is the direct connection between the two models: the SOM's trained neuron weight vectors serve as the RBF centers, replacing naive random selection with a principled, topology-aware placement.

The final model achieved an R^2 of 0.571 on the held-out test set, meaningfully outperforming a baseline that would simply predict the mean spending score ($R^2 = 0$). This validates the effectiveness of the SOM-guided center selection strategy.

6.2 Learning Outcomes

- Understood the competitive learning mechanism of SOM and how the U-Matrix reveals cluster structure.
- Implemented the Gaussian RBF kernel and understood the role of gamma in controlling activation spread.
- Recognized and actively prevented data leakage in the feature engineering step.
- Understood why topology-preserving centers (from SOM) are superior to random center selection for RBF networks.
- Applied proper evaluation using MSE, RMSE, and R^2 with a train/test split.

6.3 Future Work

- Increase SOM training iterations to 500–1000 for better convergence and lower quantization error.
- Experiment with different grid sizes (e.g. 7×7 or 10×10) and compare quantization errors.
- Replace Linear Regression with Ridge Regression to reduce potential overfitting.
- Encode the Gender feature and evaluate whether it improves prediction accuracy.
- Add a U-Matrix visualization with cluster boundary annotations to the report.
- Compare the hybrid SOM+RBF approach against a standalone k-means+RBF baseline to quantify the benefit of using SOM centers.

7. References

[1] **T. Kohonen**, "Self-organized formation of topologically correct feature maps," *Biological Cybernetics, vol. 43, pp. 59–69, 1982.*

[2] **D. Broomhead & D. Lowe**, "Multivariable functional interpolation and adaptive networks," *Complex Systems, vol. 2, pp. 321–355, 1988.*

[3] **J. Moody & C. Darken**, "Fast learning in networks of locally tuned processing units," *Neural Computation, vol. 1, no. 2, pp. 281–294, 1989.*

[4] **M. Vettigli**, *MiniSom: Minimalistic and NumPy-based implementation of the Self Organizing Map,* GitHub, 2018. [Online]. Available: https://github.com/JustGlowing/minisom

[5] **Kaggle**, *Mall Customer Segmentation Dataset.* [Online]. Available: https://www.kaggle.com/vjchoudhary7/customer-segmentation-tutorial

[6] **F. Pedregosa et al.**, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.*

Appendix: Complete Source Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from minisom import MiniSom
from scipy.spatial.distance import cdist
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
df = pd.read_csv('Mall_Customers.csv')

# Preprocessing
x = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
```

```
scaler = MinMaxScaler()
x_scaled = scaler.fit_transform(x)

# Build and train SOM
som = MiniSom(x=5, y=5, input_len=3, sigma=1.0, learning_rate=0.5)
som.random_weights_init(x_scaled)
som.train_random(x_scaled, 100)

# Assign cluster labels
clusters = [som.winner(x)[0]*5 + som.winner(x)[1] for x in x_scaled]
df['Cluster'] = clusters

# Prepare RBF features (NO data leakage: Spending Score excluded)
X_rbf = df[['Age', 'Annual Income (k$)', 'Cluster']]
y = df['Spending Score (1-100)']
scaler_rbf = MinMaxScaler()
X_rbf_scaled = scaler_rbf.fit_transform(X_rbf)

# Extract SOM neuron weights as RBF centers (KEY HYBRID STEP)
centers = som.get_weights().reshape(-1, 3)  # (25, 3)

# Apply RBF transformation
def rbf(X, centers, gamma=5.0):
    return np.exp(-gamma * cdist(X, centers)**2)

X_features = rbf(X_rbf_scaled, centers)  # (200, 25)

# Train/test split & model
x_train, x_test, y_train, y_test = train_test_split(
    X_features, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(x_train, y_train)
predictions = model.predict(x_test)

# Evaluate
mse  = mean_squared_error(y_test, predictions)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, predictions)
print(f'MSE : {mse:.2f}')
print(f'RMSE: {rmse:.2f}')
print(f'R2  : {r2:.4f}')
```